

UFiT - Universal Fieldline Tracer

Summary

This package is intended to trace magnetic field lines and calculate the squashing factor Q . It is intended to read the outputs from a number of astrophysical codes, currently: `ARMS`, `Lare3d` and `DUMFRIC`; it saves the result in a self-contained manner. Also included is a related tool `USlip` to calculate the “slippage rate” of fieldlines, such as due to nonideal effects.

Quick start

Example 1: (command line) Analytic B

Obtain all the source code and execute the following command in the terminal:

```
make
```

Ensure that you have Python installed, as well as the `numpy` and `matplotlib` libraries. Run the following script:

```
python Prepare_Spherical_Example.py
```

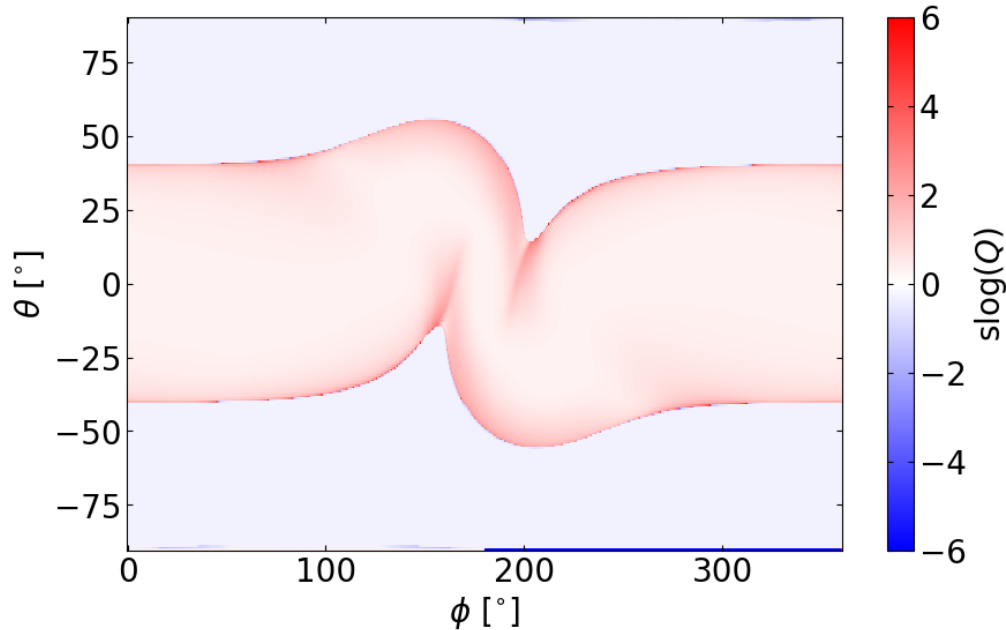
This will create a sample B -field on a grid from an analytical formula. It will also create an input file, containing the start points of all the fieldlines. Now, run UFiT itself by executing the following command:

```
./Run_UFiT -g 1 -pp -nb -sq -b Example.bin -i Example.inp -o Example.flf
```

This might take a minute or so. If all goes well, run the following script:

```
python Visualize_Spherical_Example.py
```

You should see the following map of $\text{slog}(Q)$, the signed logarithm of the squashing factor (positive/red indicates closed and negative/blue indicates open field lines).



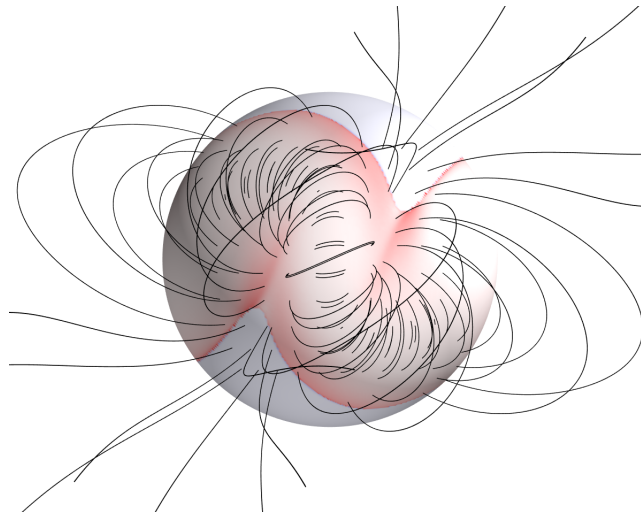
(Optional) If you have the Python library `mayavi` installed, you can then execute the following command:

```
./Run_UFiT -g 1 -pp -nb -sf -b Example.bin -i Example2.inp -o Example2.flf
```

This will use the same B -field as before, but calculate the fieldlines from a smaller set of start points; however, it will save the entire curve for each fieldline. Run the following script:

```
python Visualize_Spherical_Example3D.py
```

You should see the following 3D rendering in `mayavi` of $\text{slog}(Q)$ on a surface and a selection of black fieldlines coming from it.



Example 2: (Python call) Synoptic Magnetogram

This example requires a few additional modules. Besides the usual, you will need to have the Python modules `wget` and `astropy` installed. You can do this by setting up a virtual environment *etc*, or executing the following (you may need to have administrator rights or add the suffix `--user`):

```
pip install astropy
```

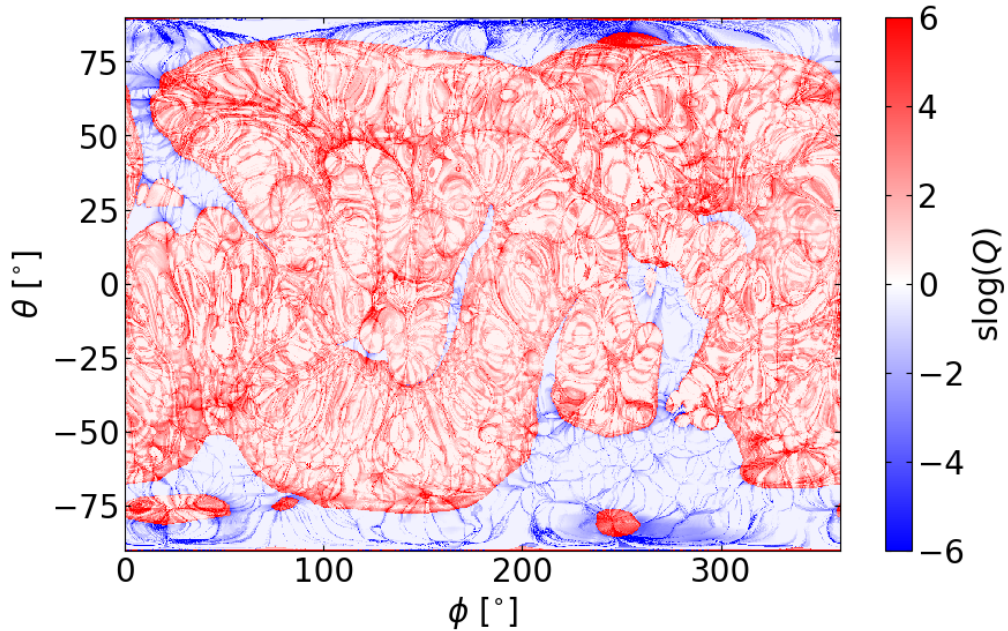
Open the script `HMI_Example.py` for editing. Near the top, you should see the following line:

```
UFiT_home_directory='/change/this/path'
```

This should be set to a path to where UFiT is installed. Now run the script by executing:

```
python HMI_Example.py
```

A synoptic magnetogram will be downloaded and a Potential Field Source Surface (PFSS) model be used to calculate B . This time, Python will call a compiled library directly, without the go-between of the input files and so on. You should see the following figure:



The squashing factor is unsurprisingly busy, because the realistic data has not been filtered. Note that the original magnetogram has been downsampled for the purposes of the PFSS extrapolation and NaNs in the original data (in the poles where data is not available) replaced with zeros.

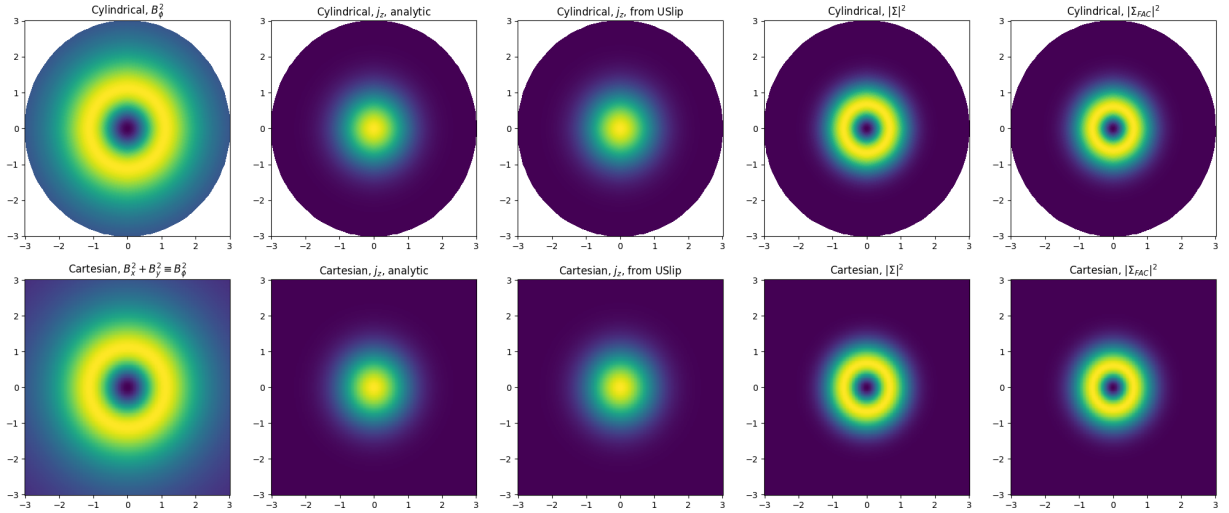
Example 3: (Python call) Fieldline slippage for a Z-pinch

Open the script `Gaussian_Current_USlip.py` for editing. As in the example above, you should change the home directory and run the script by executing:

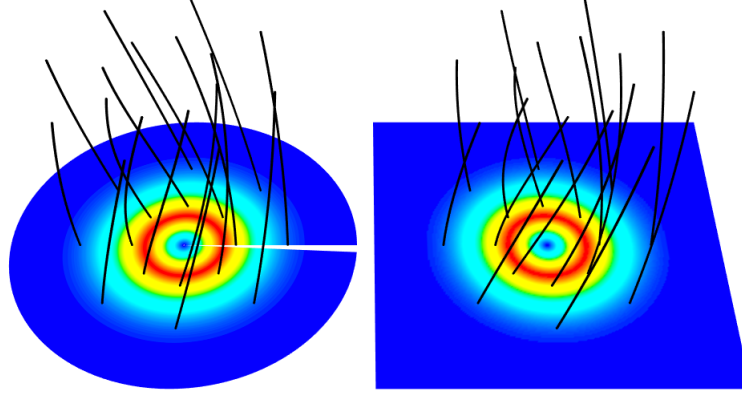
```
python Gaussian_Current_USlip.py
```

The script will generate a B -field on a grid, corresponding to something like a Z-pinch experiment. A Gaussian current profile flows in the $\hat{z}$ direction producing its own annular field through $\vec{j} = \nabla \times \vec{B}$, in addition to a constant field applied also in the $\hat{z}$ direction. The result is a helical field around this axis, such as might be seen in a tokamak.

The `USlip` routines will process this B -field to compute the current density $\vec{j}$, the fieldline slippage rate $\vec{\Sigma}$, and the slippage from field-aligned currents $\vec{\Sigma}_{FAC}$. In fact, both a Cartesian and cylindrical version of this field is computed and processed. You should see the following figure, summarizing these quantities for both coordinate systems:



(Optional) If you have the Python library `mayavi` installed, you may set the variable `visualize_3D=True` and re-run the script. This will produce the following figure:



The flat planes show $|\vec{\Sigma}|^2$ similarly to the figure above, and UFiT has been called to generate the helical fieldlines mentioned. Note that the cylindrical plane has not been correctly “wrapped”, for visualization purposes. The appropriate array could be extended by copying the values $\vec{\Sigma}(\phi = 0) \rightarrow \vec{\Sigma}(\phi = 2\pi)$. Nonetheless, the underlying calculation used the `Periodic_PHI` condition and correctly assumed this; this is merely a visual artifact.

Installation

Once the source code has been obtained, execute the following command to build the code:

```
make
```

This will follow the instructions in the `Makefile` to compile UFiT. If you wish to read DUMFRIC files directly, you will need to build UFiT with `netCDF` (instead, you can also use a Python script to convert them to a binary format first). Execute the following after substituting for the correct paths to the `netCDF` headers and library:

```
make USE_NC=True NC_INCLUDE=/include/path NC_LIB=/lib/path
```

You can omit the arguments `NC_INCLUDE` and `NC_LIB` to use the defaults, or set them within the makefile directly.

Running UFiT from the command line

Once this is done, you can run the executable as follows:

```
./Run_UFiT
```

You will need to add some command-line arguments from the table below:

Argument	Effect
-g	geometry: 0 = Cartesian; 1 = Spherical; 2 = Cylindrical
-r	print available resources (CPUs for now)
-np	number of processors (CPU)
-cs	check start points
-px	periodic X
-py	periodic Y
-pz	periodic Z
-pp	periodic ϕ
-gs	grid separate or staggered
-se	save fieldline endpoints
-sf	save fieldlines
-sq	save squashing factor Q
-sc	save connectivity type
-ud	user defined calculation
-ue	user endpoints (when you only want to define how to handle endpoints)
-nb	normalized B for Q calculation; this is highly recommended
-dl	step size
-ms	max steps
-is	integration scheme: 1 = 1 st -order; 4 = RK4;
-i	full path to input file (defining fieldline starts), <i>e.g.</i> ./dir/ufit.inp
-c	full path to command file <i>e.g.</i> ./dir/ufit.dat
-bt	B file type (see below)
-b	full path to B file <i>e.g.</i> ./dir/ufit.bin
-o	full path to output file, <i>e.g.</i> ./dir/out.flf

Add any combination of these arguments, as long as you include the correct option afterwards (*e.g.* remembering to put -o followed by a space and then the path to the output file). The order of the arguments doesn't matter. For example, the command

```
./Run_UFiT -g 1 -pp -nb -sq -b B01.bin -o Q01.flf
```

would calculate Q in the spherical geometry (periodic longitude ϕ), reading the magnetic field from B01.bin and writing the result to Q01.flf.

(Optional) Running from a command file

When running from command line, the option `-c` can be used to specify a command file, from which the arguments can be read instead of through the command line. If no command line options are supplied, UFiT will automatically look for the default command file `ufit.dat` in the current working directory. The command file overwrites any command line arguments, if there is a conflict.

Each line in the command file specifies one argument, beginning with one of the following command codes, a semicolon and a space.

Code	Effect
G:	geometry: 0 = Cartesian; 1 = Spherical; 2 = Cylindrical
R:	print available resources (CPUs for now)
NP:	number of processors (CPU)
CS:	check start points
PX:	periodic X
PY:	periodic Y
PZ:	periodic Z
PP:	periodic ϕ
GS:	grid separate or staggered
SE:	save fieldline endpoints
SF:	save fieldlines
SQ:	save squashing factor Q
SC:	save connectivity type
UD:	user defined calculation
UE:	user endpoints
NB:	normalized B for Q calculation; this is highly recommended
DL:	step size
MS:	max steps
IS:	integration scheme: 1 = 1 st -order; 4 = RK4;
I:	full path to input file (defining fieldline starts), <i>e.g.</i> <code>./dir/ufit.inp</code>
BT:	B file type (see below)
B:	full path to B file <i>e.g.</i> <code>./dir/ufit.bin</code>
O:	full path to output file, <i>e.g.</i> <code>./dir/out.flf</code>

Setting the fieldline start points

You will need to create an input file specifying the fieldline starts (sometimes called seeds). By default, it is called `ufit.inp` and contained in the current working directory; for any

other name or location, specify a valid path to it using the input file argument, *e.g.* `-i ./somewhere/starts.something`

There are several input types which allow the fieldline starts to be specified in a convenient way.

Input type	Meaning
0	The coordinate of each start point is explicitly specified
1	Regular grid of start points where coordinates in each dimension are explicitly defined
2	Regular and equally spaced grid of start points in all dimensions; only a start and end point is specified for each dimension

The first line of the input file must read

Input type: X

where X must be one of the integers above. Thereafter, the structure depends on this type.

For Input type: 0 you must then put a single integer for the number of start points on the next line. On each subsequent line, you must put three comma-separated coordinates (X, Y, Z or r, θ, ϕ respectively). For example, in the following file three very different starts are specified:

```
Input type: 0
3
0.3,0.0,0.1
0.7,0.2,0.3
0.5,-0.1,-5.0
```

For Input type: 1 you must put three integers (each one on a new line) for the number of points in each dimension (X, Y, Z or r, θ, ϕ respectively). On each of the following lines, you must put comma-separated coordinates (as many as you have previously specified). For example, in the following file there are three points in X (if Cartesian), four in Y and one in Z ; a total of twelve field lines will be traced.

```
Input type: 1
3
4
1
-1.0,0.0,1.0
```

-0.5,0.5,1.5,2.5

10.0

For **Input type:** 2 you must put three integers (each one on a new line) for the number of points in each dimension (X, Y, Z or r, θ, ϕ respectively). On each of the following lines, you must put comma-separated

For example, in the following file there are 15 points in X (if Cartesian), 10 in Y and one in Z for a total of 150 field lines. The fieldline starts will be spaced equally in X from -1 to 1 and so on.

Input type: 2

15

10

1

-1.0,1.0

0.0,1.0

0.1,0.1

In all cases, any more lines than are strictly necessary will be ignored. After making a valid input file as above, you may make several linebreaks and write some comments, store a previous configuration and so on. Only the first valid block will be used.

Supported B files

UFiT is intended to read the magnetic field from common MHD codes directly. By default, the type of file will be determined inherently from its extension (or start in the case of a `flicks/bfield` file from ARMS). Otherwise, you can use the command line argument `-bt` to specify which code created it, if you have changed the extension for example. The following table shows the currently supported file types and the code which goes after the argument `-bt` to manually specify them:

-bt	Default filename	Associated code
-1		Detect automatically
0	.bin	UFI _T specific
10	.nc	DUMFRIC
20	.sdf	Lare3d
30	flicks.	ARMS
31	bfield.	ARMS

If you have the magnetic field from some other source, you can always save it in the default UFI_T format using the Python function `write_B_file()` (see below), or create a function in another language with the Python function as a reference.

Regular vs irregular grids

We define a “regular” 3-dimensional grid of points as one made up of an outer product of 3 individual vectors, $\vec{v}_1 \otimes \vec{v}_2 \otimes \vec{v}_3$, such that each vector gives points in its respective dimension and the set of grid points is every possible combination of elements from the three vectors. In other words, this is the ordinary bunch of cuboids you think of when a 3-dimensional grid comes to mind. Note that the spacing between the points does not need to be constant, or identical, and nor does the number of points in each dimension need to be identical. This is the preferred way to input the magnetic field to UFI_T.

Separate (or staggered) grids

It is normally assumed that the B -field is defined on a single 3-dimensional grid, namely that at every grid point a vector $\vec{B}$ is completely known. A “separate” grid is one where each of the components of B is defined at differing spatial locations. There are effectively three separate grids, which may have different numbers and separations of points. This means that the procedure for the interpolation of B must be performed separately for each component.

Using Python

Python for plotting and analysis

UFI_T is distributed along with a set of Python scripts. All the relevant functions and required libraries are contained in `UFIT_Functions_Python.py`. You will find example scripts for plotting and reading the UFI_T outputs. Near the top of each of these is the following line

```
UFIT_home_directory='/change/this/path'
```

which must be changed to instead contain a path to where the file `UFIT_Functions_Python.py` is kept. This way you can copy and modify the specific scripts for each task while keeping the main function library in one place (where you can also occasionally modify it).

(Optional) Calling routines directly from Python

You may create a Python script which calls compiled routines in `UFiT_Python_Callable.so` directly, without the go-between of calling from command line and writing in/outputs to disk. Example 2 above features just such a script.

User defined calculations

You are, of course, free to change the source code as you see fit. The easiest way to run your own bespoke calculation might be to use the `-user_defined` or `-ud` option. You will then need to modify the file

`UFiT_User_Functions.F90`

which by default contains five subroutines:

1. `prepare_user_defined` - called once initially, this gives you the opportunity to allocate memory, make preliminary calculations or read in additional files
2. `fieldline_classification_user` - specify how the type of fieldline, *e.g.* open vs closed, should be classified
3. `trace_cartesian_user` - main tracing routine which integrates forwards and back from a fieldline start if Cartesian coordinates are selected
4. `trace_spherical_user` - as above, for sphericals; feel free to leave this alone if you don't intend to use sphericals and vice versa
5. `trace_cylindrical_user` - as above, for cylindricals

The default version calculates $|\mathbf{B}|$ at each end of the field line and then saves the expansion factor $|\mathbf{B}_+|^2/|\mathbf{B}_-|^2$ as the user quantity. Any end quantity can be easily calculated by changing this directly. A quantity, such as a sum or a maximum, along field lines must be calculated by adding lines inside the two `do while` loops. Define and allocate additional variables as required.

User defined endpoints

The type of fieldline is saved directly when running with the `-save_connection` or `-sc` option, and indirectly by the sign of Q when that is output.

Visualization Routines

Description coming soon!TM

USlip

This tool is automatically compiled by `make`, exactly the same way as the rest of `UFiT`. You can run it from command line by executing:

```
./Run_USlip
```

You will need to add some command-line arguments from the table below:

Argument	Effect
-g	geometry: 0 = Cartesian; 1 = Spherical; 2 = Cylindrical
-r	print available resources (CPUs for now)
-np	number of processors (CPU)
-px	periodic X
-py	periodic Y
-pz	periodic Z
-pp	periodic ϕ
-gs	grid separate or staggered
-c	full path to command file <i>e.g.</i> <code>./dir/ufit.dat</code>
-bt	B file type (as <code>UFiT</code>)
-b	full path to B file <i>e.g.</i> <code>./dir/ufit.bin</code>
-od	full path to output directory, <i>e.g.</i> <code>./dir/</code>

Likewise, you can specify a command file

Note that, unlike `UFiT`, the command line and the `Python` versions, when running with `write_output = False`, will not output a bespoke file, but instead will create a copy with the new data appended. For example, `DUMFRIC` outputs will create a new `netCDF` file with new arrays; `ARMS` outputs will have new variables added to their adaptive grids and so on. You must therefore provide the argument `-od` with a fresh directory to write this output. Alternatively, use the `Python` interface, as in Example 3 above.

Note: you must provide the appropriate type of slash at the end of the output directory, as the name of the new output file will be appended directly at the end of whatever you put. For instance,

```
./Run_USlip -b abc.xyz -od ./mydir/
```

will produce a new file in a new directory, such that its path is `./mydir/abc.xyz`, while

```
./Run_USlip -b abc.xyz -od ./new
```

will produce a file called `newabc.xyz` in the working directory. This is a feature, not a bug! You may, for example, want to label the new files with a prefix; then, simply do not terminate your output directory with a slash.

Important! Do not overwrite the original file, for example by setting the output directory to be where the input files are stored.